

UNIVERSIDAD POLITÉCNICA --- DE SAN LUIS POTOSÍ

Actividad 12: File Path Traversal Validation Of File Extension With Null Byte Bypass Nivel-Practitioner

López Gómez Samuel-177393

CNO IV – Seguridad Informática

28/09/26

Servando López Contreras

Índice

Contenido

Índice	2
2. Marco teórico y explicación técnica de la vulnerabilidad	3
3. Procedimiento paso a paso y evidencias	4
3.1 Preparación del entorno de intercepción con Burp Suite	4
3.2 Acceso al laboratorio y generación de la instancia	6
3.3 Envío de una petición a Repeater	8
3.4 Intentos de explotación en Burp Repeater	10
3.5 Confirmación de la resolución del laboratorio	11
4. Explicación técnica de los payloads	12
4.1 Payload de control — 57.jpg	12
4.2 Payload fallido — /etc/passwd	12
4.3 Payload exitoso — ../../../../etc/passwd%00.png	12
5. Mitigación recomendada	12

2. Marco teórico y explicación técnica de la vulnerabilidad

Como en los laboratorios anteriores de la serie Path Traversal, la aplicación carga las imágenes de sus productos mediante un endpoint del tipo:

```
GET /image?filename=57.jpg
```

El mecanismo de defensa de este laboratorio en particular consiste en validar que el nombre de archivo recibido termine ("ends with") con una extensión permitida, por ejemplo .jpg o .png. La idea detrás de este control es que, aunque el atacante incluya secuencias de traversal al inicio o en medio del valor, el archivo final al que se pretenda acceder siempre deberá ser una imagen, lo cual en teoría limitaría el impacto de cualquier intento de path traversal.

El problema de este enfoque radica en cómo se procesa la cadena en distintas capas del sistema. Un byte nulo, representado en la petición HTTP como %00 (y que, una vez decodificado, corresponde al carácter con valor 0x00), tiene un significado especial en muchos lenguajes de bajo nivel y en numerosas funciones del sistema operativo basadas en cadenas de estilo C: dichas funciones consideran que la cadena termina exactamente en ese byte, ignorando por completo cualquier carácter que venga después.

Esto genera una discrepancia entre dos capas de la aplicación que interpretan la misma cadena de forma distinta:

- La lógica de validación de extensión, escrita típicamente en un lenguaje de alto nivel, evalúa la cadena completa tal como la recibe (por ejemplo, ../../etc/passwd%00.png) y, al comprobar el final de esa cadena completa, encuentra la extensión .png, por lo que la validación se considera superada.
- La función del sistema operativo o de la biblioteca de bajo nivel que finalmente abre el archivo en disco, en cambio, trata el byte nulo como el verdadero final de la cadena, por lo que abre el archivo ../../etc/passwd, descartando silenciosamente todo lo que sigue al %00 (incluida la extensión .png que "engañó" a la validación).

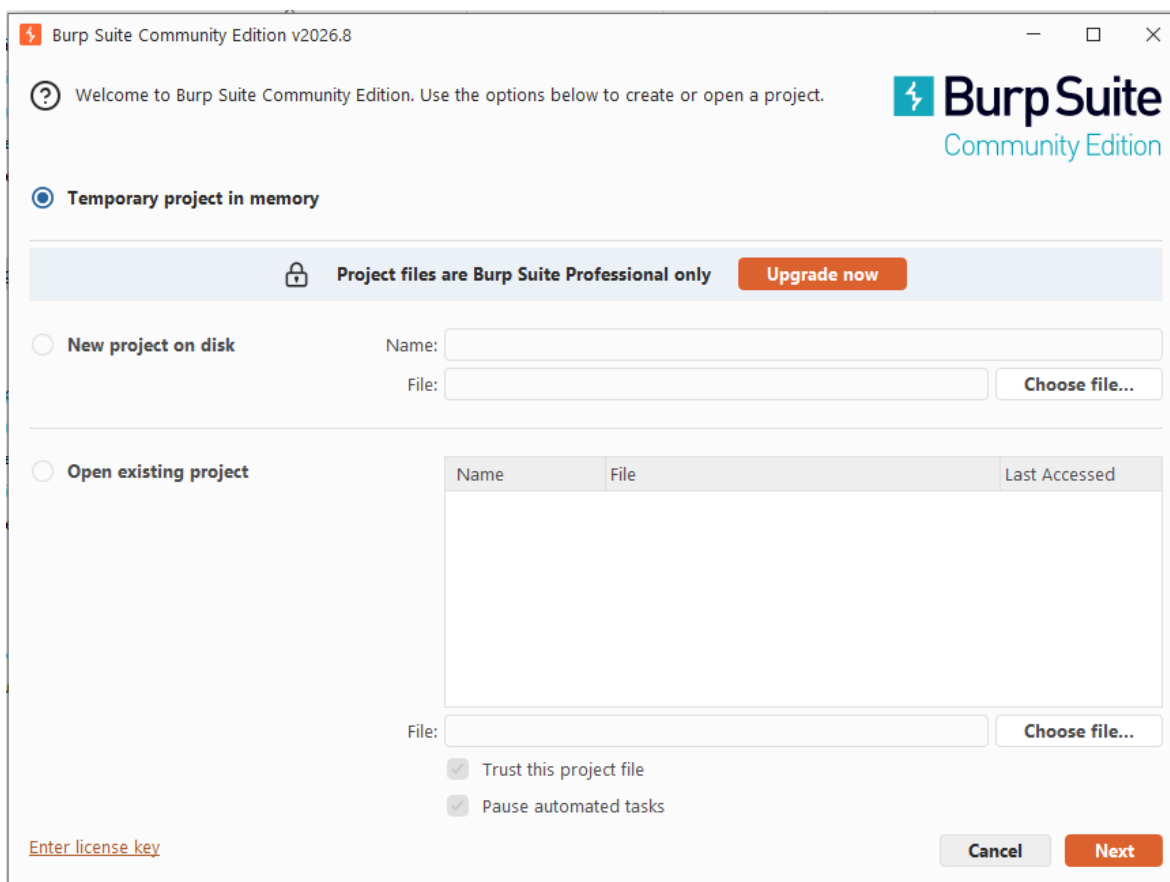
El resultado es que la aplicación cree haber validado correctamente que solo se accederá a un archivo de imagen, mientras que, en la práctica, el archivo realmente abierto es uno completamente distinto y arbitrario, elegido por el atacante mediante las secuencias de traversal que preceden al byte nulo. Este defecto se documenta específicamente como CWE-626 (Null Byte Interaction Error, comúnmente conocido como Poison Null Byte), y suele presentarse en aplicaciones que combinan un lenguaje de aplicación de alto nivel con bibliotecas o llamadas al sistema operativo escritas en C, donde persiste esta diferencia de interpretación de las cadenas.

Desde la perspectiva de los principios de seguridad, este caso refuerza la importancia de validar la entrada exactamente en la misma forma y capa en que finalmente será utilizada: una validación realizada sobre una representación de la cadena distinta a la que efectivamente procesará el sistema de archivos no ofrece ninguna garantía real de seguridad, sin importar cuán correcta parezca la lógica de validación en sí misma.

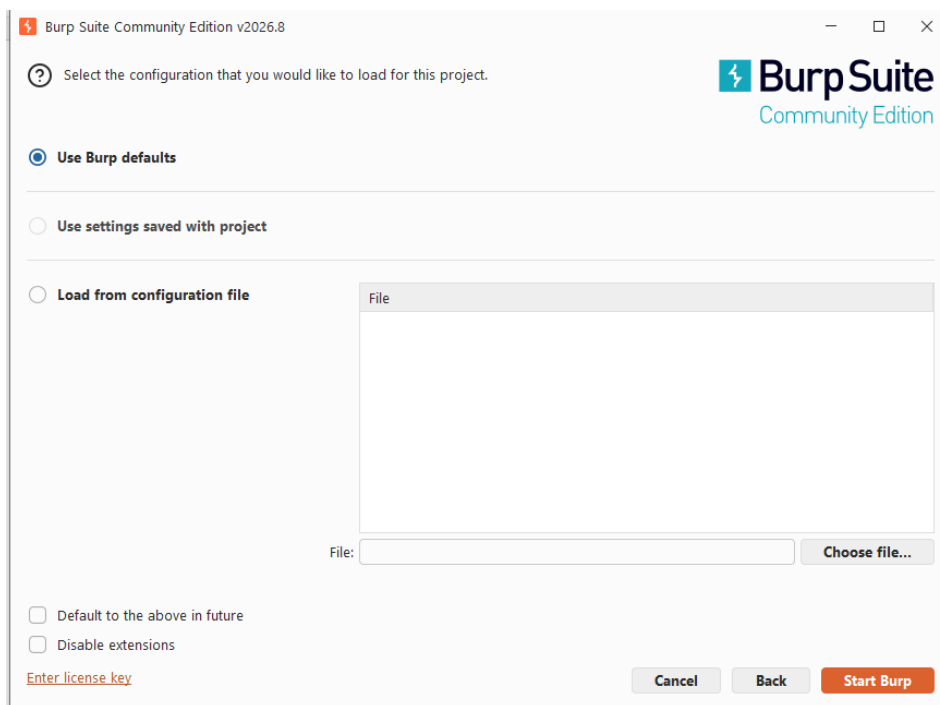
3. Procedimiento paso a paso y evidencias

3.1 Preparación del entorno de intercepción con Burp Suite

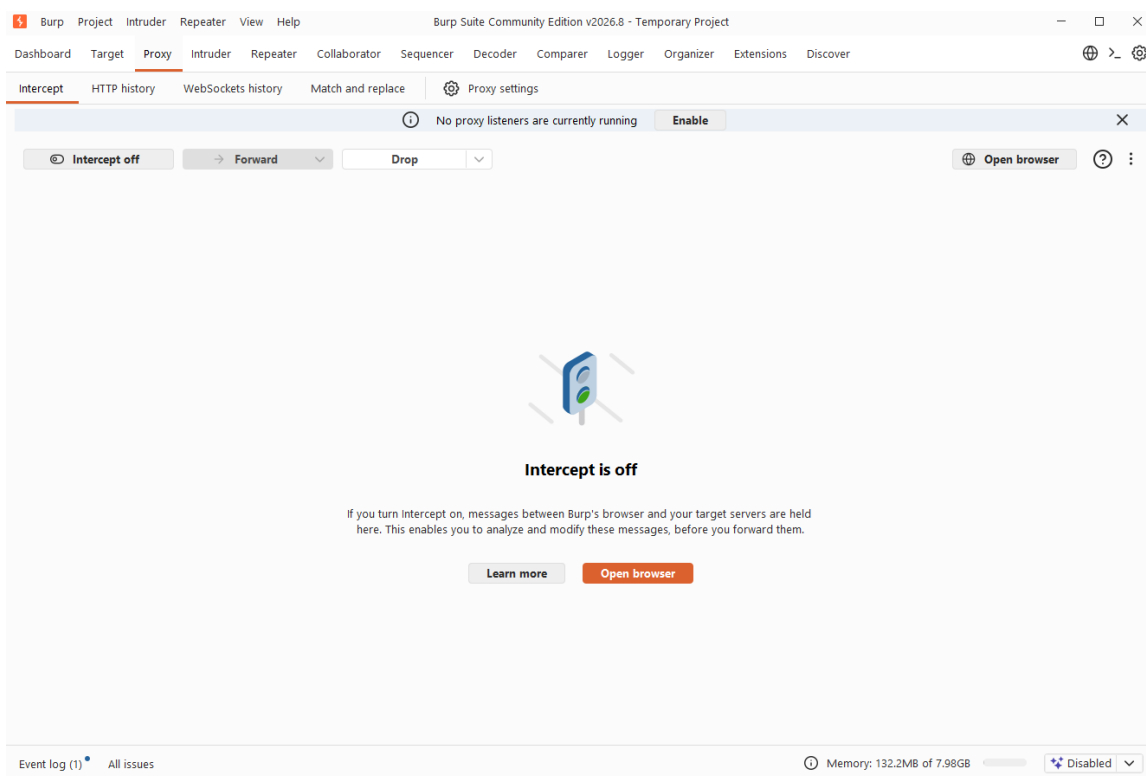
Paso 1. se abrió Burp Suite Community Edition y, en la ventana de bienvenida, se seleccionó "Temporary project in memory", adecuada para una sesión de práctica puntual que no requiere persistirse en disco.



Paso 2. en la siguiente ventana se eligió "Use Burp defaults" y se presionó "Start Burp", inicializando la herramienta con su configuración estándar

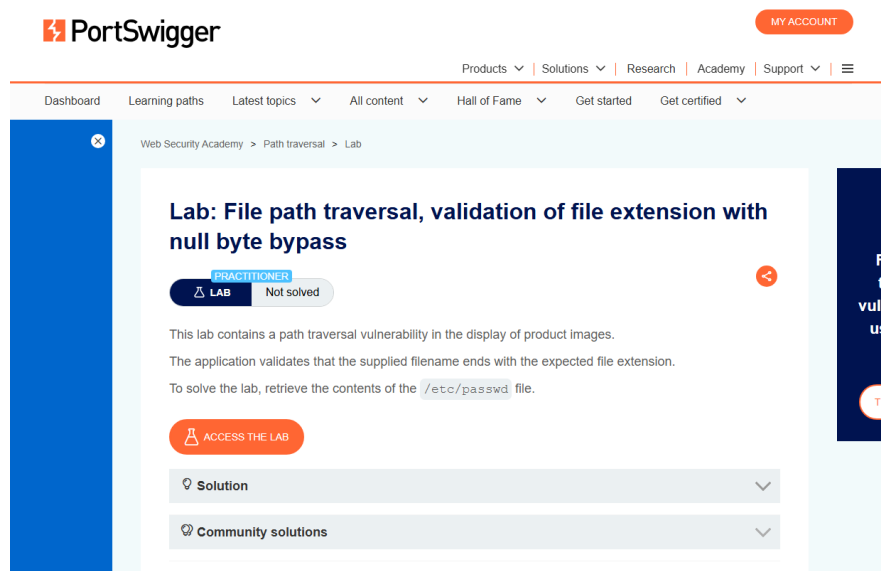


Paso 3. Burp abrió su panel principal en la pestaña Proxy > Intercept, con el interceptor apagado, estado normal antes de comenzar a capturar tráfico.

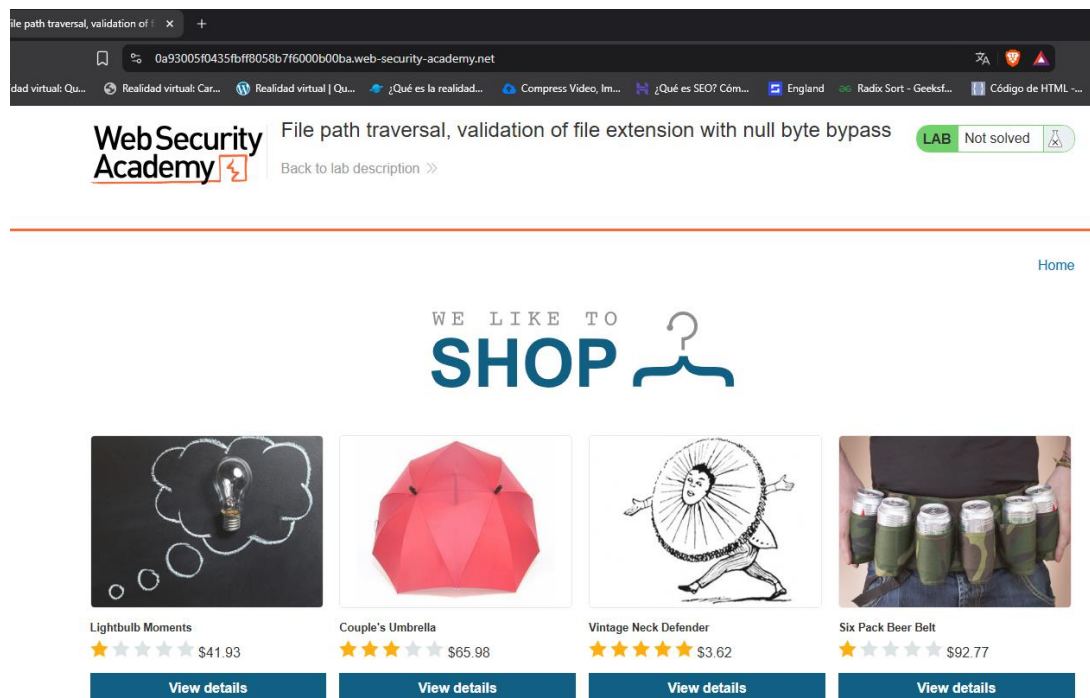


3.2 Acceso al laboratorio y generación de la instancia

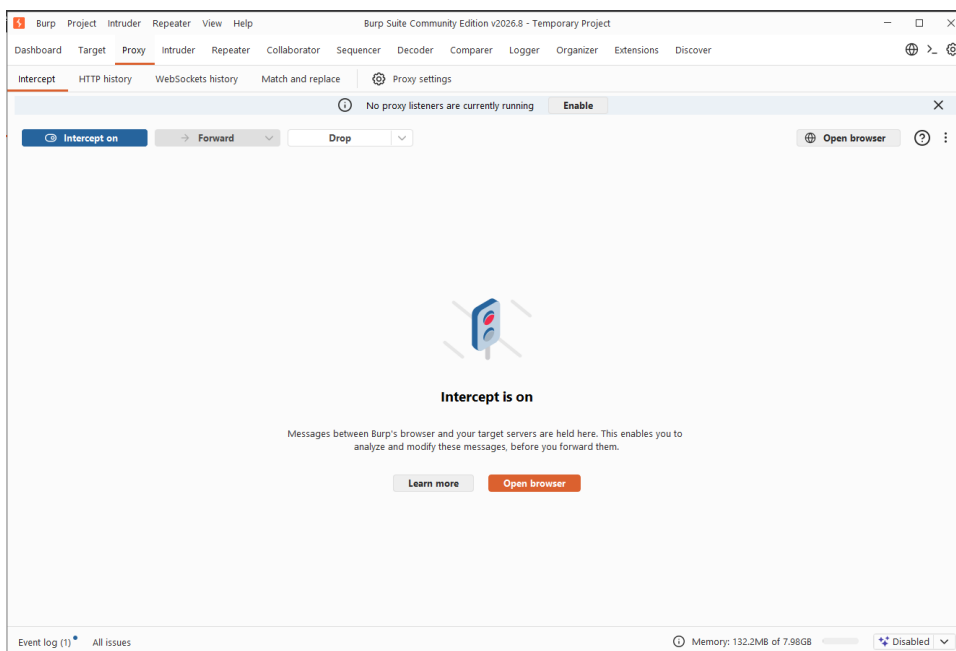
Paso 4. desde el navegador predeterminado se accedió a la página de descripción del laboratorio en PortSwigger Web Security Academy. En ella se explica que la aplicación valida que el nombre de archivo suministrado termine con la extensión esperada, y que el objetivo es recuperar /etc/passwd. Se dio clic en "ACCESS THE LAB" para generar la instancia.



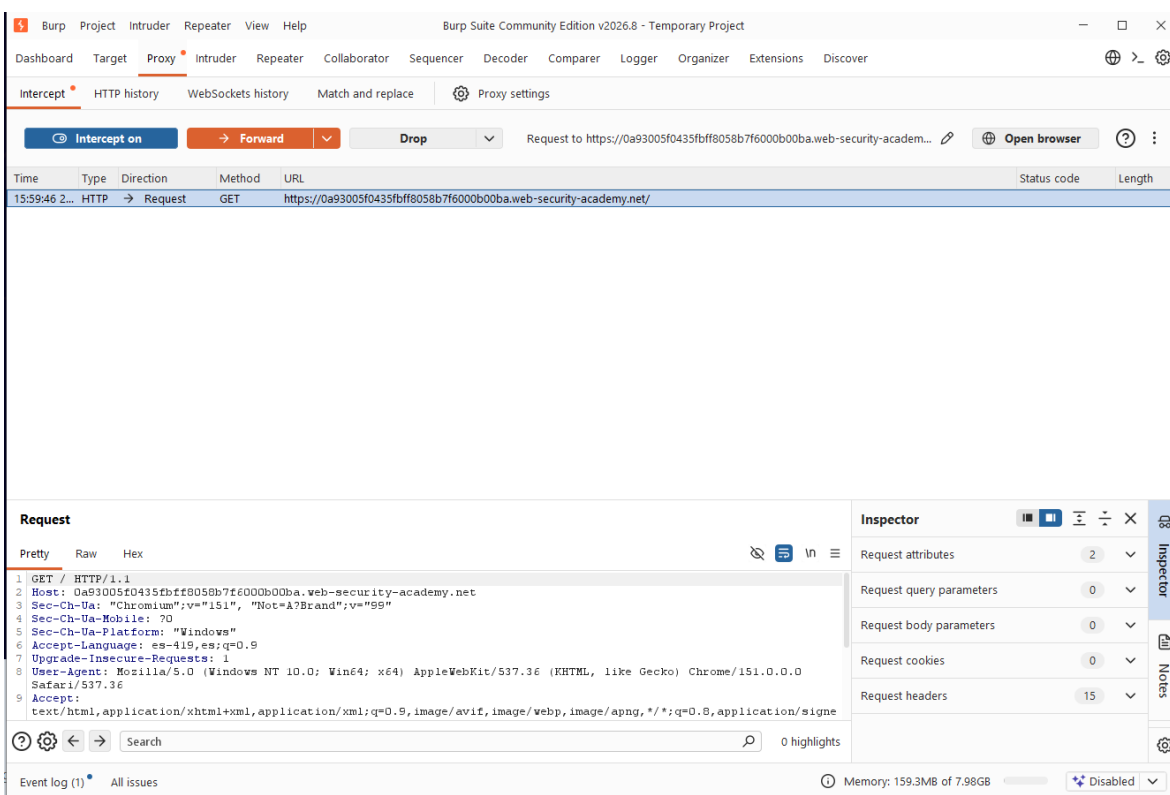
Paso 5. el navegador redirigió a la instancia única del laboratorio, con una URL aleatoria asociada a la sesión, mostrando la tienda "We Like To Shop" en estado "Not solved". De la barra de direcciones se copió dicha URL para reutilizarla en el navegador integrado de Burp.



Paso 6. se regresó a Burp Suite y se activó "Intercept on" en la pestaña Proxy, de modo que toda petición quedara detenida hasta reenviarse manualmente. Después se utilizó "Open browser" para abrir la instancia de Chromium preconfigurada por Burp con el proxy y el certificado CA correctos.



Paso 7. en el navegador embebido de Burp se pegó la URL de la instancia obtenida en el paso 5. Al cargar la página, Burp interceptó de inmediato la primera petición: GET / dirigida al dominio de la instancia, correspondiente al documento HTML principal. Se presionó "Forward" para permitir que la carga continuara.



Paso 8. al seguir reenviando peticiones, Burp fue deteniendo, una a una, las numerosas solicitudes GET /image?filename=N.jpg generadas automáticamente para cargar las miniaturas del catálogo de productos (filename=57.jpg, 28.jpg, 11.jpg, 31.jpg, etc.), confirmando que en este laboratorio el parámetro filename vuelve a ser un simple nombre de archivo, sin ruta completa.

The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' section is active, showing a list of intercepted HTTP requests. The first request is highlighted, showing its details in the 'Request' pane below.

Time	Type	Direction	Method	URL
15:59:59 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=57.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=28.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=11.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=31.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=38.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=61.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=70.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=21.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=45.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=65.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=18.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=24.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=75.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=5.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=53.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=74.jpg
16:00:00 2...	HTTP	→ Request	GET	https://0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net/image?filename=7.jpg

Request

Pretty Raw Hex

```
1 GET /image?filename=57.jpg HTTP/2
2 Host: 0a93005f0435fbff8058b7f6000b00ba.web-security-academy.net
3 Cookie: session=TkwLW5AodLX8mvpOE4GoQqX7N1s217YM
4 Sec-Ch-UA-Platform: "Windows"
5 Accept-Language: es-419,es;q=0.9
6 Sec-Ch-UA: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-UA-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
```

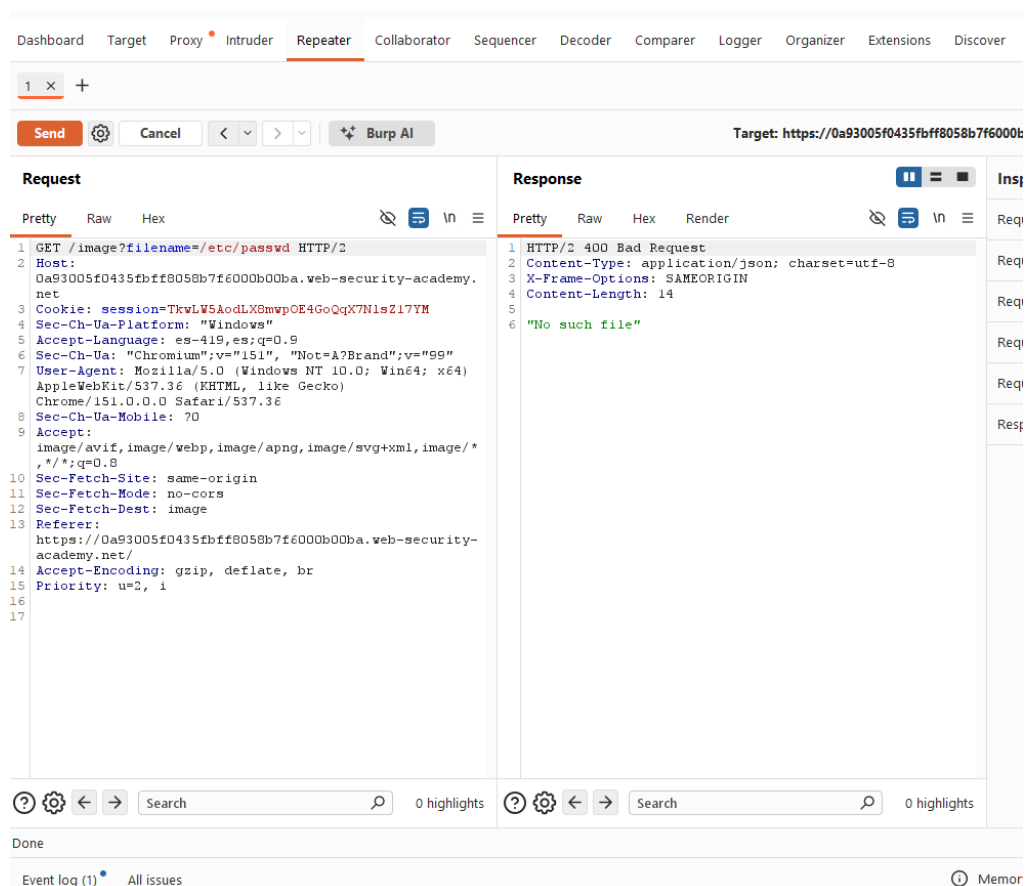
Event log (1) All issues

3.3 Envío de una petición a Repeater

Paso 9. sobre una de estas peticiones (filename=57.jpg) se dio clic derecho y se seleccionó "Send to Repeater" (Ctrl+R), enviando una copia exacta a la pestaña Repeater para su manipulación manual.

3.4 Intentos de explotación en Burp Repeater

Paso 11. como primer intento se reemplazó el valor del parámetro por filename=/etc/passwd, sin ninguna extensión de archivo. El servidor respondió 400 Bad Request con el mensaje "No such file", confirmando que la validación de extensión (o la resolución de la ruta en ausencia de una extensión válida) rechaza este tipo de entrada de inmediato.



Paso 12. como segundo intento se construyó el payload filename=../../etc/passwd%00.png, combinando una secuencia de traversal relativo con un byte nulo (%00) seguido de la extensión .png, exigida por la validación. Al enviar la petición, el servidor respondió 200 OK con Content-Type: image/png, pero el cuerpo de la respuesta contenía texto plano: el listado completo de cuentas del sistema operativo del servidor (root, daemon, bin, sys, www-data, backup, peter, carlos, entre otras), es decir, el contenido real de /etc/passwd.

4. Explicación técnica de los payloads

4.1 Payload de control — 57.jpg

```
filename=57.jpg
```

Nombre de archivo legítimo perteneciente al catálogo de productos, utilizado para confirmar el comportamiento normal del endpoint antes de cualquier intento de explotación.

4.2 Payload fallido — /etc/passwd

```
filename=/etc/passwd
```

No contiene ninguna extensión reconocida por la validación (no termina en .jpg, .png ni similar), por lo que la aplicación rechaza la petición de inmediato con 400 Bad Request. Este intento confirmó la existencia del control de validación de extensión anunciado en el enunciado del laboratorio.

4.3 Payload exitoso — ../../../../etc/passwd%00.png

```
filename=../../../../etc/passwd%00.png
```

- La secuencia ../../../../ al inicio realiza el recorrido de traversal habitual, retrocediendo tres niveles de directorio desde la carpeta base de imágenes hasta alcanzar la raíz del sistema de archivos, desde donde se referencia directamente etc/passwd.
- El carácter %00 representa, una vez decodificado por el servidor, un byte nulo (0x00) insertado justo después de etc/passwd y antes de la extensión .png.
- La cadena .png colocada al final de todo el payload no tiene como propósito acceder realmente a un archivo con ese nombre; su único objetivo es hacer que la validación de "la cadena debe terminar en una extensión permitida" se cumpla, ya que esa validación examina la cadena completa, incluyendo todo lo posterior al byte nulo.
- Sin embargo, cuando el servidor utiliza esa misma cadena para abrir físicamente el archivo en el sistema operativo, la función subyacente interpreta el byte nulo como el final real de la cadena, y por lo tanto abre efectivamente ../../../../etc/passwd, ignorando por completo el byte nulo y la extensión .png que lo siguen. El resultado es la lectura arbitraria del archivo /etc/passwd, a pesar de que la aplicación "vio" una extensión válida durante su validación.

5. Mitigación recomendada

- Rechazar explícitamente cualquier entrada que contenga bytes nulos (%00 o su equivalente decodificado 0x00) antes de realizar cualquier otra validación o de utilizar el valor para acceder al sistema de archivos.
- Actualizar y mantener actualizado el entorno de ejecución (lenguaje, runtime y bibliotecas), ya que la mayoría de los lenguajes y frameworks modernos ya bloquean o manejan de forma

segura los bytes nulos embebidos en cadenas al interactuar con el sistema operativo; este tipo de bypass es característico de versiones antiguas o de bibliotecas nativas de bajo nivel.

- Validar la extensión del archivo únicamente después de canonicalizar la ruta completa (resolviendo cualquier secuencia de traversal y eliminando caracteres de control), y no sobre la cadena cruda recibida del cliente.
- Sustituir el nombre de archivo controlado por el usuario por un identificador indirecto (un ID numérico de producto), de modo que el servidor construya la ruta real y la extensión sin depender de ningún dato proporcionado directamente por el cliente.
- Adoptar un enfoque de lista blanca (whitelist) estricta sobre el nombre completo del archivo (por ejemplo, únicamente dígitos seguidos de una extensión permitida), en lugar de validar solamente el sufijo de la cadena recibida.